Pandas manipulation and transform:

1. **Renaming Columns**
2. **Changing Data Types**
3. **Applying Functions (map, apply, applymap)**
4. **Sorting Data**
5. **Handling Duplicates**
6. **Pivoting and Melting Data**
7. **Working with Text Data (String Methods)**
8. **Working with Date and Time Data**
9. **Binning Data (Categorization)**
10. **Reshaping with Stack/Unstack**

Let's start by setting up a sample DataFrame that we can manipulate.

```
import pandas as pd
import numpy as np

# Create a more complex sample DataFrame for manipulation
data = {
    'OrderID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    'CustomerID': ['C101', 'C102', 'C101', 'C103', 'C104', 'C102', 'C105',
'C101', 'C106', 'C103'],
    'Product': ['Laptop', 'Mouse', 'Keyboard', 'Monitor', 'Laptop', 'Keyboard',
'Mouse', 'Monitor', 'Laptop', 'Keyboard'],
    'Price': [1200.00, 25.50, 75.00, 300.00, 1150.00, 80.00, 22.00, 320.00,
1300.00, 68.00],
    'Quantity': [1, 2, 1, 1, 1, 2, 3, 1, 1, 1],
    'OrderDate': ['2023-01-15', '2023-01-16', '2023-01-15', '2023-01-17',
'2023-01-18', '2023-01-16', '2023-01-19', '2023-01-20', '2023-01-21',
'2023-01-17'],
    'Discount_Code': ['SALE10', np.nan, 'SALE10', 'FREESHIP', np.nan,
'SALE10', np.nan, 'FREESHIP', 'SUMMER20', np.nan],
    'Rating': [4.5, 3.8, 4.0, 4.2, 4.7, 3.5, 3.9, 4.1, 4.6, 3.7]
}
df = pd.DataFrame(data)

print("--- Original DataFrame ---")
print(df)
```

```
print("\nDataFrame Info:")
df.info()
print("\n")
```

1. Renaming Columns

You might want to change column names for clarity or consistency.

Python

```
# Method 1: Using a dictionary to rename specific columns
df_renamed = df.rename(columns={'Price': 'Unit_Price', 'Quantity':
'Order_Quantity'})
print("--- 1.1 Renaming specific columns ---")
print(df_renamed.head())

# Method 2: Renaming all columns by assigning a new list (order
matters)
new_columns = ['Order_ID', 'Customer_ID', 'Product_Name',
'Unit_Price', 'Quantity_Ordered',
               'Date_of_Order', 'Promo_Code', 'Product_Rating']
df.columns = new_columns # Modifies original df
print("\n--- 1.2 Renaming all columns by assigning a list ---")
print(df.head())
print("\n")
```

2. Changing Data Types (`astype()`)

Ensuring correct data types is crucial for efficient operations and preventing errors.

Python

```
# Convert 'Order_ID' and 'Customer_ID' to object/string type (if they weren't already)
# Convert 'Date_of_Order' to datetime objects
# Convert 'Unit_Price' to float64 for consistent decimal operations

df['Order_ID'] = df['Order_ID'].astype(str)
df['Customer_ID'] = df['Customer_ID'].astype(str)
df['Date_of_Order'] = pd.to_datetime(df['Date_of_Order']) # More robust for dates
df['Product_Rating'] = df['Product_Rating'].astype(float) # Already float, but good example

print("--- 2. Changing Data Types ---")
print(df.info())
print("\n")
```

3. Applying Functions (`map()`, `apply()`, `applymap()`)

These methods allow you to apply custom logic to your data.

- **`map()` (Series only):** Element-wise application, often used for mapping values from a dictionary or Series.
- **`apply()` (Series/DataFrame):** Can apply a function element-wise on a Series, or row/column-wise on a DataFrame.
- **`applymap()` (DataFrame only):** Element-wise application across the entire DataFrame (deprecated for new code, prefer `df.map` for element-wise on DataFrame in newer pandas versions).

Python

```
# --- 3.1 Using .map() for element-wise transformation on a Series ---
# Map product names to categories
product_categories = {
    'Laptop': 'Electronics',
    'Mouse': 'Peripherals',
```

```
    'Keyboard': 'Peripherals',
    'Monitor': 'Peripherals'
}
df['Product_Category'] = df['Product_Name'].map(product_categories)
print("--- 3.1 Adding 'Product_Category' using .map() ---")
print(df[['Product_Name', 'Product_Category']].head())

# --- 3.2 Using .apply() on a Series for element-wise transformation ---
# Round the product rating to 1 decimal place
df['Product_Rating_Rounded'] = df['Product_Rating'].apply(lambda x:
round(x, 1))
print("\n--- 3.2 Rounding 'Product_Rating' using .apply() on Series ---")
print(df[['Product_Rating', 'Product_Rating_Rounded']].head())

# --- 3.3 Using .apply() on a DataFrame (row-wise) ---
# Calculate Total Amount for each order
df['Total_Amount'] = df.apply(lambda row: row['Unit_Price'] *
row['Quantity_Ordered'], axis=1)
print("\n--- 3.3 Calculating 'Total_Amount' using .apply() row-wise ---")
print(df[['Unit_Price', 'Quantity_Ordered', 'Total_Amount']].head())

# Add a derived column based on conditional logic
def get_order_status(row):
    if row['Total_Amount'] > 1000:
        return 'High Value'
    elif row['Total_Amount'] > 100:
        return 'Medium Value'
    else:
        return 'Low Value'

df['Order_Value_Status'] = df.apply(get_order_status, axis=1)
print("\n--- 3.3 Adding 'Order_Value_Status' using .apply() row-wise with
conditional logic ---")
print(df[['Total_Amount', 'Order_Value_Status']].head())

# Note: For simple element-wise operations, vectorized operations are
usually faster than .apply()
df['Discounted_Price'] = df['Unit_Price'] * 0.9 # This is much faster than
apply(lambda x: x*0.9)
print("\n")
```

4. Sorting Data (`sort_values()`, `sort_index()`)

Organize your data based on column values or index labels.

Python

```
# Sort by a single column
df_sorted_price = df.sort_values(by='Total_Amount', ascending=False)
print("--- 4.1 Sorted by 'Total_Amount' (descending) ---")
print(df_sorted_price.head())

# Sort by multiple columns
df_sorted_multi = df.sort_values(by=['Customer_ID', 'Date_of_Order'])
print("\n--- 4.2 Sorted by 'Customer_ID' then 'Date_of_Order' ---")
print(df_sorted_multi.head())

# Sort by index
df_sorted_index = df.sort_index(ascending=False) # Not as useful here as default index is numeric
print("\n")
```

5. Handling Duplicates (`drop_duplicates()`, `duplicated()`)

Identify and remove duplicate rows.

Python

```
# Create a DataFrame with duplicates
df_dup = pd.DataFrame({
    'col1': ['A', 'B', 'A', 'C', 'B'],
    'col2': [1, 2, 1, 3, 2],
```

```
    'col3': [10, 20, 10, 30, 25]
})
print("--- 5.1 Original DataFrame with duplicates ---")
print(df_dup)

# Check for duplicates
print("\n--- 5.2 Duplicated rows (returns Boolean Series) ---")
print(df_dup.duplicated()) # By default considers all columns

# Drop duplicates (keeps first occurrence by default)
df_no_duplicates_all = df_dup.drop_duplicates()
print("\n--- 5.3 DataFrame after dropping all duplicate rows ---")
print(df_no_duplicates_all)

# Drop duplicates based on a subset of columns
df_no_duplicates_subset = df_dup.drop_duplicates(subset=['col1',
'col2'])
print("\n--- 5.4 DataFrame after dropping duplicates based on 'col1' and
'col2' ---")
print(df_no_duplicates_subset)

# Keep the last occurrence of duplicates
df_no_duplicates_last = df_dup.drop_duplicates(keep='last')
print("\n--- 5.5 DataFrame after dropping duplicates (keeping last) ---")
print(df_no_duplicates_last)
print("\n")
```

6. Pivoting and Melting Data

- **pivot_table():** Reshapes data based on values in columns, performing aggregation if multiple values fall into the same cell. Great for creating summary tables.
- **melt():** Transforms wide format data into long format data. Useful for converting columns into row values.

Python

```
# --- 6.1 Pivot Table ---
# Calculate total quantity and average price per product and customer
pivot_df = df.pivot_table(
    values=['Quantity_Ordered', 'Total_Amount'],
    index='Product_Name',
    columns='Customer_ID',
    aggfunc={'Quantity_Ordered': 'sum', 'Total_Amount': 'mean'}
)
print("--- 6.1 Pivot Table (Total Quantity & Avg Total Amount per Product by Customer) ---")
print(pivot_df)
print("\n")

# --- 6.2 Melting Data ---
# Convert 'Unit_Price' and 'Total_Amount' columns into rows
df_melted = df.melt(
    id_vars=['Order_ID', 'Customer_ID', 'Product_Name'],
    value_vars=['Unit_Price', 'Total_Amount'],
    var_name='Metric_Type',
    value_name='Metric_Value'
)
print("--- 6.2 Melted DataFrame (converting price/amount columns to rows) ---")
print(df_melted.head())
print(df_melted.tail())
print("\n")
```

7. Working with Text Data (String Methods `.str`)

Pandas provides a `.str` accessor to apply string methods to Series of string type.

Python

```
# Convert 'Product_Name' to uppercase
df['Product_Name_Upper'] = df['Product_Name'].str.upper()
print("--- 7.1 'Product_Name' in uppercase ---")
```

```
print(df[['Product_Name', 'Product_Name_Upper']].head())

# Check if 'Product_Name' contains 'Mouse'
df['Is_Mouse'] = df['Product_Name'].str.contains('Mouse')
print("\n--- 7.2 Checking if 'Product_Name' contains 'Mouse' ---")
print(df[['Product_Name', 'Is_Mouse']].head())

# Replace values in 'Promo_Code'
df['Promo_Code_Clean'] = df['Promo_Code'].str.replace('SALE10',
'SALE_10%', na_rep='NO_DISCOUNT')
print("\n--- 7.3 Replacing values and handling NaNs in 'Promo_Code'
---")
print(df[['Promo_Code', 'Promo_Code_Clean']].head(7))

# Get length of 'Customer_ID'
df['Customer_ID_Length'] = df['Customer_ID'].str.len()
print("\n--- 7.4 Length of 'Customer_ID' ---")
print(df[['Customer_ID', 'Customer_ID_Length']].head())
print("\n")
```

8. Working with Date and Time Data (.dt accessor)

After converting a column to datetime objects (using pd.to_datetime()), you can use the .dt accessor to extract various date/time components.

Python

```
# Ensure 'Date_of_Order' is datetime type (already done in step 2)
# df['Date_of_Order'] = pd.to_datetime(df['Date_of_Order'])

print("--- 8. Working with Date and Time Data ---")

# Extract year, month, day, day of week
df['Order_Year'] = df['Date_of_Order'].dt.year
df['Order_Month'] = df['Date_of_Order'].dt.month
df['Order_Day'] = df['Date_of_Order'].dt.day
```

```
df['Order_Day_of_Week'] = df['Date_of_Order'].dt.day_name() # Get day name

print(df[['Date_of_Order', 'Order_Year', 'Order_Month', 'Order_Day', 'Order_Day_of_Week']].head())

# Calculate difference between dates (if you had another date column)
# For example, let's create a 'DeliveryDate' column (fictional)
df['DeliveryDate'] = df['Date_of_Order'] + pd.to_timedelta(np.random.randint(1, 5, len(df)), unit='D')
df['Delivery_Time_Days'] = (df['DeliveryDate'] - df['Date_of_Order']).dt.days
print("\n--- 8.1 Calculating Delivery Time in Days ---")
print(df[['Date_of_Order', 'DeliveryDate', 'Delivery_Time_Days']].head())
print("\n")
```

9. Binning Data (Categorization)

`pd.cut()` and `pd.qcut()` are used to segment and sort data values into bins (intervals).

- **`pd.cut()`:** Bins data into a fixed number of bins or specified bin edges. Useful when you have predefined ranges.
- **`pd.qcut()`:** Bins data into quantiles, so each bin has roughly the same number of observations. Useful for creating balanced groups.

Python

```
# --- 9.1 Binning using pd.cut() (equal-width bins or custom bins) ---
# Create age groups based on predefined bins
bins = [0, 50, 100, 1500] # Price ranges
labels = ['Low Price', 'Medium Price', 'High Price']
df['Price_Category'] = pd.cut(df['Unit_Price'], bins=bins, labels=labels, right=True) # right=True means bins are (a, b]
print("--- 9.1 Binning 'Unit_Price' into categories using pd.cut() ---")
print(df[['Unit_Price', 'Price_Category']].head())

# --- 9.2 Binning using pd.qcut() (equal-frequency bins/quantiles) ---
```

```
# Bin 'Total_Amount' into 4 quantiles
df['Total_Amount_Quartile'] = pd.qcut(df['Total_Amount'], q=4,
labels=['Q1', 'Q2', 'Q3', 'Q4'])
print("\n--- 9.2 Binning 'Total_Amount' into Quartiles using pd.qcut() ---")
print(df[['Total_Amount',
'Total_Amount_Quartile']].sort_values(by='Total_Amount').head())
print(df['Total_Amount_Quartile'].value_counts()) # Check distribution
print("\n")
```

10. Reshaping with Stack/Unstack

These methods are used to pivot and unpivot data based on index levels. They are most useful when dealing with MultiIndex Series or DataFrames.

- **stack():** "Stacks" the columns of a DataFrame into rows, resulting in a MultiIndex Series or DataFrame.
- **unstack():** "Unstacks" a level of the (MultiIndex) index to form new columns.

Python

```
# Create a DataFrame with a MultiIndex for demonstration
df_multi = df.set_index(['Customer_ID', 'Order_ID'])
df_multi = df_multi[['Total_Amount', 'Quantity_Ordered',
'Product_Rating']]
print("--- 10.1 Original DataFrame with MultiIndex ---")
print(df_multi.head())

# --- Stacking ---
# Convert columns to rows, creating a new index level
stacked_series = df_multi.stack()
print("\n--- 10.2 Stacked Series (columns become inner-most index) ---")
print(stacked_series.head(9))
print("Type after stack:", type(stacked_series))

# --- Unstacking ---
# Unstack the 'Product_Name' level from the index to columns (requires
'Product_Name' to be in index)
```

```
df_unstack_example = df.set_index(['Customer_ID', 'Product_Name'])
df_unstack_example = df_unstack_example[['Total_Amount',
'Quantity_Ordered']]

unstacked_df = df_unstack_example.unstack(level='Product_Name')
print("\n--- 10.3 Unstacked DataFrame (Product_Name moved from
index to columns) ---")
print(unstacked_df.head())
print("\n")

# You can unstack different levels
unstacked_df_level0 = df_unstack_example.unstack(level=0) # Unstack
Customer_ID
# print("\nUnstacked by Customer_ID (level 0):\n",
unstacked_df_level0.head())
```